

MULTIMODAL COGNITIVE ASSISTANT

Complete Build Guide — RTX 4060 Edition — v3

Trust State Machine · Local RAG · File Ingestion API · Web UI · No Docker Required

File	Purpose	Status
trust_state.py	Intent classification + trust scoring	Existing
llm_engine.py	Ollama / Mistral chat with trust integration	Existing
ingest.py	Chunk text + embed + store in local Qdrant	Existing
test_json_mode.py	Verify Mistral JSON output mode	Existing
retrieval.py	Search local Qdrant by query vector	NEW
file_handler.py	Extract text from PDF / DOCX / CSV / TXT	NEW
main.py	FastAPI server: /upload /chat /files routes	NEW
ui.html	Browser UI: drag-drop upload + chat interface	NEW

STEP 1 — Install All Required Libraries

Run this single command in your project terminal (inside your .venv):

```
pip install ollama fastapi uvicorn python-multipart qdrant-client \
sentence-transformers tiktoken pypdf python-docx
```

Library	Version	Why You Need It
ollama	latest	Python client to talk to local Ollama / Mistral
fastapi	>=0.110	Web framework — powers /upload /chat /files endpoints
uvicorn	latest	ASGI server that runs FastAPI
python-multipart	latest	Required for FastAPI file uploads (UploadFile)
qdrant-client	>=1.9	Local file-based vector DB — no Docker needed
sentence-transformers	latest	Loads all-MiniLM-L6-v2 for text embeddings
tiktoken	latest	Token counting for smart text chunking
pypdf	>=4.0	Extracts text from PDF files
python-docx	latest	Extracts text from .docx Word files

NOTE: sentence-transformers will download the all-MiniLM-L6-v2 model (~90 MB) on first run. It is cached locally after that.

NOTE: Ollama must be installed separately from <https://ollama.com> — it is NOT a pip package.

Ollama Setup (one-time):

```
# 1. Download and install Ollama from https://ollama.com
# 2. Pull the Mistral model (first run only ~ 4 GB download)
ollama pull mistral:7b-instruct-q4_K_M
# 3. Verify it works
ollama list
# Should show: mistral:7b-instruct-q4_K_M
```

STEP 2 — Project File Structure

```
mca_project/
├── .venv/                # your virtual environment
├──
├── trust_state.py        # (existing) intent + trust score
├── llm_engine.py         # (existing) Mistral chat engine
├── ingest.py             # (existing) chunk + embed + store
├── test_json_mode.py     # (existing) JSON mode test
├──
├── retrieval.py          # (NEW) search local vector DB
├── file_handler.py       # (NEW) extract text from files
├── main.py               # (NEW) FastAPI server + routes
├── ui.html               # (NEW) browser drag-drop UI
├──
├── local_qdrant/         # auto-created by qdrant-client
│   └── collection/personal_memory/ # your vectors stored here
```

★ NEW | NEW FILE — retrieval.py

What it does: embeds your query with the same model used during ingest, then searches the local Qdrant DB and returns the top-5 most relevant text chunks as context.

```
from qdrant_client import QdrantClient
from sentence_transformers import SentenceTransformer
DB_PATH = './local_qdrant'
COLLECTION_NAME = 'personal_memory'
TOP_K = 5

client = QdrantClient(path=DB_PATH)
encoder = SentenceTransformer('all-MiniLM-L6-v2')

def retrieve(query: str) -> str:
    query_vector = encoder.encode(query).tolist()
    results = client.search(
        collection_name=COLLECTION_NAME,
        query_vector=query_vector,
        limit=TOP_K
    )
    if not results:
        return ''
    context_parts = []
    for r in results:
        text = r.payload.get('text', '')
        source = r.payload.get('source', 'unknown')
        score = round(r.score, 3)
        context_parts.append(f'[source: {source} | score: {score}]\n{text}')
    return '\n\n'.join(context_parts)
```

★ NEW | NEW FILE — file_handler.py

What it does: auto-detects the uploaded file extension and extracts plain text from PDF, DOCX, CSV, and TXT files.

```
import os, csv
SUPPORTED_TYPES = {'.txt', '.pdf', '.docx', '.csv'}
def extract_text(file_path: str) -> str:
    ext = os.path.splitext(file_path)[1].lower()
    if ext not in SUPPORTED_TYPES:
        raise ValueError(f'Unsupported type: {ext}')
    if ext == '.txt':
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    elif ext == '.pdf':
        from pypdf import PdfReader
        reader = PdfReader(file_path)
        return '\n'.join(page.extract_text() or '' for page in reader.pages)
    elif ext == '.docx':
        from docx import Document
        doc = Document(file_path)
        return '\n'.join(p.text for p in doc.paragraphs if p.text.strip())
    elif ext == '.csv':
        rows = []
        with open(file_path, 'r', encoding='utf-8') as f:
            for row in csv.reader(f):
                rows.append(', '.join(row))
        return '\n'.join(rows)
```

★ NEW | NEW FILE — main.py (FastAPI Server)

What it does: starts a local web server on port 8000 with three key endpoints and serves the browser UI automatically.

API Endpoints:

Method	Endpoint	What it does
GET	/	Opens the browser UI (drag-drop upload + chat)
POST	/upload	Accepts a file, extracts text, ingests into Qdrant
POST	/chat	RAG query: retrieve context, ask Mistral, return answer
GET	/files	Lists all previously uploaded and ingested files
GET	/health	Health check — confirms server is alive

```
import os, shutil, json, datetime
from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from ingest import ingest_knowledge
from retrieval import retrieve
from llm_engine import chat as llm_chat
from file_handler import extract_text, SUPPORTED_TYPES

app = FastAPI(title='Multimodal Cognitive Assistant')
app.add_middleware(CORSMiddleware, allow_origins=['*'],
                  allow_methods=['*'], allow_headers=['*'])

UPLOAD_DIR = './uploaded_files'
os.makedirs(UPLOAD_DIR, exist_ok=True)
ingested_files = []

@app.get('/', response_class=HTMLResponse)
async def serve_ui():
    return HTMLResponse(content=open('ui.html').read())

@app.post('/upload')
async def upload_file(file: UploadFile = File(...)):
    ext = os.path.splitext(file.filename)[1].lower()
    dest = os.path.join(UPLOAD_DIR, file.filename)
    with open(dest, 'wb') as f:
        shutil.copyfileobj(file.file, f)
    text = extract_text(dest)
    txt_path = dest + '_extracted.txt'
    with open(txt_path, 'w') as f: f.write(text)
    ingest_knowledge(txt_path)
    ingested_files.append({'filename': file.filename, ...})
    return {'status': 'success', 'word_count': len(text.split())}

@app.post('/chat')
async def chat_endpoint(req: ChatRequest):
    context = retrieve(req.text)
    return llm_chat(req.text, context=context, history=req.history)

if __name__ == '__main__':
    import uvicorn
    uvicorn.run('main:app', host='0.0.0.0', port=8000, reload=True)
```

★ NEW | NEW FILE — ui.html (Browser Interface)

What it does: a dark-themed single-page web app automatically served by FastAPI. No install needed — just open your browser.

Features:

- Drag and drop or click-to-browse file upload (PDF, DOCX, CSV, TXT)
- Shows upload progress and word count after ingestion
- Displays list of all ingested files in sidebar
- Chat box with Enter-to-send and multi-line support
- Shows trust score, response mode, and intent profile per reply
- Maintains conversation history for multi-turn dialogue

How to open:

```
# After running python main.py, open your browser and go to:  
http://localhost:8000  
# That's it — no separate frontend server needed.
```

STEP 3 — How to Run Everything

Terminal 1 — Start Ollama

```
ollama serve
```

Terminal 2 — Start the MCA Server

```
cd mca_project
.venv\Scripts\activate # Windows
# or
source .venv/bin/activate # Mac / Linux
python main.py
# Server starts at http://localhost:8000
```

Browser — Open the UI

```
http://localhost:8000
```

Full Data Flow:

- 1. Upload a PDF/DOCX/CSV/TXT via the browser UI
- 2. file_handler.py extracts plain text from the file
- 3. ingest.py chunks the text and embeds it with all-MiniLM-L6-v2
- 4. Vectors are stored in ./local_qdrant (no Docker, fully local)
- 5. Type a question in the chat box
- 6. retrieval.py searches Qdrant for the top-5 relevant chunks
- 7. llm_engine.py sends the question + context to Mistral
- 8. trust_state.py scores the intent and adjusts the trust level
- 9. The answer, trust score, and intent appear in the chat UI

Troubleshooting

Common Errors

- **ModuleNotFoundError: No module named 'fastapi'** — Run the pip install command in Step 1 inside your .venv
- **Connection refused on port 8000** — Make sure python main.py is running in a separate terminal
- **ollama.ResponseError: model not found** — Run: ollama pull mistral:7b-instruct-q4_K_M
- **Upload returns 400 Unsupported type** — Only .pdf .docx .csv .txt are supported
- **Empty answers from chat** — No files ingested yet; upload a document first
- **CUDA out of memory** — Stop Ollama, restart, load models one at a time
- **sentence_transformers download hangs** — First run downloads ~90 MB model; wait for it
- **tiktoken encoding mismatch** — cl100k_base is GPT tokenizer; ~5% diff from Mistral is normal

VRAM Budget (RTX 4060 8 GB)

Component	VRAM	OOM Tip
Mistral-7B Q4_K_M	~4.8 GB	Switch to Q2_K (~3.5 GB) if needed
all-MiniLM-L6-v2 embed	~0.1 GB	Can also run on CPU — fast enough
Framework overhead	~0.2 GB	—
Total	~5.1 GB	Fits RTX 4060 8 GB with room to spare

Quick Execution Checklist v3

Setup	Install	<code>pip install ollama fastapi uvicorn python-multipart qdrant-client sentence-transformers tiktoken pypdf python-docx</code>
Setup	Ollama	<code>ollama pull mistral:7b-instruct-q4_K_M</code> then <code>ollama serve</code>
Files	Existing	<code>trust_state.py</code> , <code>llm_engine.py</code> , <code>ingest.py</code> already working
Files	New	Create <code>retrieval.py</code> , <code>file_handler.py</code> , <code>main.py</code> , <code>ui.html</code>
Run	Server	<code>python main.py</code> — server starts at <code>localhost:8000</code>
Use	Browser	Open <code>http://localhost:8000</code> — drag drop a file then ask questions

Multimodal Cognitive Assistant v3 — Full local stack, no Docker, browser UI included